

ZeRO-3 (FSDP): A Concrete Walkthrough with Actual Numbers

1. Our Tiny Transformer Setup (Same as ZeRO-1 & ZeRO-2)

Hidden dimension (d)

= 4

Number of attention heads

= 2 → head dimension $d_k = 4/2 = 2$

FFN inner dimension

= 16 (4× expansion)

Vocab size

= 8

Sequence length (T)

= 3 (3 tokens)

Number of GPUs

= 2 (GPU-0 and GPU-1)

ZeRO-3 is also known as **FSDP (Fully Sharded Data Parallelism)**. Along with sharding optimizer states (ZeRO-1) and gradients (ZeRO-2), we now **shard the model parameters themselves**. There is no full model replica anywhere.

2. Listing Every Parameter (with shapes)

Following the architecture diagram (LN1 → Attention → residual → LN2 → FFN → residual → W_vocab → softmax):

Layer	Name	Shape	#Elements
LayerNorm 1	γ_1	(4,)	4
	β_1	(4,)	4
Attention:			
Query weights	W_q	(4, 4)	16
Key weights	W_k	(4, 4)	16
Value weights	W_v	(4, 4)	16
Output proj	W_o	(4, 4)	16
LayerNorm 2	γ_2	(4,)	4
	β_2	(4,)	4
FFN:			
Up-projection	W_1	(4, 16)	64
Bias 1	b_1	(16,)	16
Down-projection	W_2	(16, 4)	64
Bias 2	b_2	(4,)	4
Output head	W_{vocab}	(4, 8)	32
TOTAL PARAMETERS:			260

3. Concrete Parameter Values (Initialised)

Same initial weights as ZeRO-1 and ZeRO-2 (identical starting model).

W_q (Query weights) — shape (4, 4):

```
W_q = [ 0.12  0.34 -0.21  0.05 ]  
      [-0.15  0.22  0.11 -0.08 ]  
      [ 0.30 -0.10  0.18  0.27 ]  
      [-0.05  0.14 -0.33  0.09 ]
```

W_k (Key weights) — shape (4, 4):

```
W_k = [ 0.20 -0.11  0.07  0.15 ]  
      [ 0.03  0.28 -0.14  0.10 ]  
      [-0.22  0.06  0.31 -0.05 ]  
      [ 0.17 -0.09  0.02  0.24 ]
```

W_v (Value weights) — shape (4, 4):

```
W_v = [ 0.08  0.19 -0.12  0.25 ]  
      [-0.07  0.33  0.04 -0.16 ]  
      [ 0.21 -0.03  0.15  0.11 ]  
      [-0.14  0.08  0.26 -0.02 ]
```

W₁ (FFN up-projection) — shape (4, 16): (showing first 4 of 16 columns)

```
W_1 = [ 0.10  0.05 -0.22  0.13  ... ]  (4 rows × 16 cols = 64 values)  
      [-0.08  0.17  0.03 -0.11  ... ]  
      [ 0.25 -0.06  0.14  0.09  ... ]  
      [-0.03  0.21 -0.07  0.18  ... ]
```

W_{vocab} (output head) — shape (4, 8):

```
W_vocab = [ 0.11 -0.05  0.18  0.03 -0.14  0.22 -0.09  0.07 ]  
          [-0.06  0.13  0.02 -0.17  0.08  0.04  0.20 -0.11 ]  
          [ 0.15 -0.02  0.09  0.21 -0.08  0.12 -0.03  0.16 ]  
          [-0.10  0.07 -0.13  0.05  0.19 -0.06  0.14  0.01 ]
```

LayerNorm parameters:

```
γ1 = [1.0, 1.0, 1.0, 1.0]  (initialized to ones)  
β1 = [0.0, 0.0, 0.0, 0.0]  (initialized to zeros)
```

$\gamma_2 = [1.0, 1.0, 1.0, 1.0]$
 $\beta_2 = [0.0, 0.0, 0.0, 0.0]$

4. Memory Accounting: ZeRO-3 vs ZeRO-2 vs ZeRO-1 vs Baseline

The final frontier: **parameters themselves are partitioned**. No GPU holds a full copy of the model at rest.

What	Precision	Bytes/element
------	-----------	---------------

Parameter	BF16	2
-----------	------	---

Gradient	BF16	2
----------	------	---

Optimizer states (Adam)		
-------------------------	--	--

Master copy of parameter	FP32	4
--------------------------	------	---

First moment m	FP32	4
----------------	------	---

Second moment v	FP32	4
-----------------	------	---

Comparison (260 parameters, 2 GPUs):

	No ZeRO (per GPU)	ZeRO-1 (per GPU)	ZeRO-2 (per GPU)	ZeRO-3 (per GPU)
--	----------------------	---------------------	---------------------	---------------------

Params (BF16):	260×2= 520	260×2= 520	260×2= 520	130×2= 260 ← PARTITIONED!
----------------	------------	------------	------------	---------------------------

Grads (BF16):	260×2= 520	260×2= 520	130×2= 260	130×2= 260
---------------	------------	------------	------------	------------

Optimizer (FP32):	260×12=3120	130×12=1560	130×12=1560	130×12=1560
-------------------	-------------	-------------	-------------	-------------

TOTAL per GPU:	4,160	2,600	2,340	2,080
----------------	-------	-------	-------	-------

SAVING vs baseline:	—	37.5%	43.8%	50.0%
---------------------	---	-------	-------	-------

We've cut memory in half compared to baseline — and **everything** scales as 1/N with more GPUs.

5. The Partition: Who Owns What — At Rest

Same flat partition, but now parameters are sharded too:

Flat parameter vector (260 elements):

[$\gamma_1(4)$ | $\beta_1(4)$ | $W_q(16)$ | $W_k(16)$ | $W_v(16)$ | $W_o(16)$ | $\gamma_2(4)$ | $\beta_2(4)$ | $W_1(64)$ | $b_1(16)$ | $W_2(64)$ | $b_2(4)$ | $W_{vocab}(32)$]

indices 0..129 → GPU-0 owns

indices 130..259 → GPU-1 owns

"Owns" now means EVERYTHING — params, grads, AND optimizer:

GPU-0 stores persistently (at rest):

$\theta_0[130]$ in BF16 (parameter shard)	= 260 bytes
$m_0[130]$ in FP32 (first moments)	= 520 bytes
$v_0[130]$ in FP32 (second moments)	= 520 bytes
$p_0[130]$ in FP32 (master params)	= 520 bytes

1,820 bytes

GPU-1 stores persistently (at rest):

$\theta_1[130]$ in BF16 (parameter shard)	= 260 bytes
$m_1[130], v_1[130], p_1[130]$ in FP32	= 1,560 bytes

1,820 bytes

Neither GPU can run the model alone! They each hold only half the parameters. This is the fundamental difference from ZeRO-1/2, where every GPU had the full parameter set.

What GPU-0 actually holds at rest:

GPU-0's parameter shard (indices 0–129):

$\gamma_1 = [1.0, 1.0, 1.0, 1.0]$	(4 values)
$\beta_1 = [0.0, 0.0, 0.0, 0.0]$	(4 values)
$W_q = [\begin{matrix} 0.12 & 0.34 & -0.21 & 0.05 \\ -0.15 & 0.22 & 0.11 & -0.08 \\ 0.30 & -0.10 & 0.18 & 0.27 \\ -0.05 & 0.14 & -0.33 & 0.09 \end{matrix}]$	(16 values)
W_k (full 16), W_v (full 16), W_o (full 16)	(48 values)
γ_2 (4), β_2 (4)	(8 values)
W_1 rows 0–1 (first 2 of 4 rows $\times$ 16 cols = 32)	(32 values)
W_1 rows 2–3, first 2 cols	(2 values, to reach 130)

130 values total

GPU-0 does NOT have: remaining $W_1, b_1, W_2, b_2, W_{\text{vocab}}$
GPU-1 has those instead.

6. Walking Through One Training Step

Input

Same 3-token input:

$x_0 = [0.5, -0.3, 0.8, 0.1]$ $\leftarrow$ token 0
[0.2, 0.7, -0.1, 0.4] $\leftarrow$ token 1
[-0.6, 0.3, 0.5, -0.2] $\leftarrow$ token 2

GPU-0 gets micro-batch A, GPU-1 gets micro-batch B.

STEP 1: Forward Pass — Layer by Layer with Gather & Flush

This is the major new cost in ZeRO-3. Before computing each layer, the GPU must all-gather the full parameters for that layer from all GPUs, compute, then **flush** (discard) the gathered parameters immediately.

We group our model into logical "layers" for the gather-compute-flush pattern:

Layer 0: LN1 (γ_1, β_1) = 8 params
Layer 1: Attention (W_q, W_k, W_v, W_o) = 64 params
Layer 2: LN2 (γ_2, β_2) = 8 params
Layer 3: FFN (W_1, b_1, W_2, b_2) = 148 params
Layer 4: Output (W_{vocab}) = 32 params

Layer 0: LN1 — Gather, Compute, Flush

GATHER: All-gather γ_1 and β_1

GPU-0 has: γ_1, β_1 (these happen to be in its shard)

GPU-1 has: nothing for LN1 params

All-gather $\rightarrow$ both GPUs now temporarily hold:

$\gamma_1 = [1.0, 1.0, 1.0, 1.0]$

$\beta_1 = [0.0, 0.0, 0.0, 0.0]$

Temporary memory cost: $8 \times 2 = 16$ bytes per GPU

COMPUTE: $x_1 = \text{LN}(x_0) = \gamma_1 \odot (x_0 - \mu) / \sigma + \beta_1$

For token 0: $\mu = 0.275, \sigma \approx 0.398$

$x_1[0] = [0.565, -1.445, 1.319, -0.439]$

(identical computation to ZeRO-1/2)

FLUSH: Discard the gathered γ_1, β_1 from non-owner GPUs

GPU-0: keeps γ_1, β_1 (it's the owner)

GPU-1: frees the 16 bytes

Memory returns to baseline.

Layer 1: Attention — Gather, Compute, Flush

GATHER: All-gather W_q, W_k, W_v, W_o

GPU-0 has: W_q, W_k, W_v, W_o (all in its shard)

GPU-1 has: none of these

All-gather $\rightarrow$ both GPUs temporarily hold all 64 params

Temporary memory cost: $64 \times 2 = 128$ bytes per GPU

COMPUTE: Full attention (both GPUs, independently on their own micro-batch)

$$Q = x_1 \cdot W_q \rightarrow (3, 4)$$

$$K = x_1 \cdot W_k \rightarrow (3, 4)$$

$$V = x_1 \cdot W_v \rightarrow (3, 4)$$

For token 0:

$$\begin{aligned} Q[0] &= [0.565, -1.445, 1.319, -0.439] \times W_q \\ &= [0.703, -0.319, \dots] \end{aligned}$$

Split into heads, compute attention scores:

$$A_0 = \text{softmax}(Q_0 \cdot K_0^T / \sqrt{2}) \rightarrow (3, 3)$$

$$x_{\text{attn}} = \text{Concat}(A_0 \cdot V_0, A_1 \cdot V_1) \cdot W_o \rightarrow (3, 4)$$

FLUSH: Discard gathered W_q, W_k, W_v, W_o

GPU-0: keeps them (owner)

GPU-1: frees 128 bytes

Layer 2: LN2 — Gather, Compute, Flush

GATHER: All-gather γ_2, β_2 (8 params, 16 bytes temporary)

COMPUTE:

$$x_{\text{res1}} = x_0 + x_{\text{attn}} \rightarrow (3, 4)$$

$$x_2 = \text{LN2}(x_{\text{res1}}) \rightarrow (3, 4)$$

FLUSH: Discard γ_2, β_2 from non-owner

Layer 3: FFN — Gather, Compute, Flush

GATHER: All-gather W_1, b_1, W_2, b_2

This is the BIG layer: 148 params total

GPU-0 has: partial W_1 (first ~32 elements)

GPU-1 has: rest of W_1, b_1, W_2, b_2

All-gather $\rightarrow$ both GPUs temporarily hold all 148 params

Temporary memory: $148 \times 2 = 296$ bytes per GPU

**** THIS IS THE PEAK MEMORY MOMENT ****

Each GPU has: its 130-element shard (260 bytes)

+ full FFN params temporarily (296 bytes)

+ activations from previous layers

COMPUTE:

$x_{\text{ffn}} = \text{ReLU}(x_2 \cdot W_1 + b_1) \cdot W_2 + b_2 \rightarrow (3, 4)$

$(3,4) \times (4,16) = (3,16) \quad (3,16) \times (16,4) = (3,4)$

$x_{\text{final}} = x_{\text{res1}} + x_{\text{ffn}} \rightarrow (3, 4)$

FLUSH: Discard full W_1, b_1, W_2, b_2

GPU-0: frees the non-owned portions

GPU-1: frees the non-owned portions

Each GPU back to holding only its 130-element shard

Layer 4: Output — Gather, Compute, Flush

GATHER: All-gather W_{vocab} (32 params, 64 bytes temporary)

COMPUTE:

$\text{logits} = x_{\text{final}} \cdot W_{\text{vocab}} \rightarrow (3, 4) \times (4, 8) = (3, 8)$

$z = \text{softmax}(\text{logits}) \rightarrow (3, 8)$

$\text{Loss} = \text{CrossEntropy}(z, \text{targets}) \rightarrow \text{scalar}$

FLUSH: Discard W_{vocab} from non-owner

Forward Pass Complete. Summary of All-Gathers During Forward:

Layer	Params gathered	Temp memory	Communication
LN1	8 params	16 bytes	$8 \times 2 = 16 \text{ B}$
Attention	64 params	128 bytes	$64 \times 2 = 128 \text{ B}$
LN2	8 params	16 bytes	$8 \times 2 = 16 \text{ B}$
FFN	148 params	296 bytes	$148 \times 2 = 296 \text{ B}$
Output	32 params	64 bytes	$32 \times 2 = 64 \text{ B}$
TOTAL	260 params		$260 \times 2 = 520 \text{ B}$
↑↑↑ THIS IS NEW COMMUNICATION (ZeRO-1/2 had 0 extra comm during forward pass)			

STEP 2: Backward Pass — Layer by Layer with Gather, Compute & Flush (Reverse Order)

The backward pass mirrors the forward: for each layer (in reverse), we must all-gather the parameters again (they were flushed!), compute gradients, then flush.

Backward Layer 4: Output — Gather, Backprop, Flush

GATHER: All-gather W_{vocab} again (flushed after forward) |
64 bytes temporary |

COMPUTE BACKWARD:
GPU-0: $g_{\text{vocab}}^{\text{(A)}} = \partial \text{Loss}_A / \partial W_{\text{vocab}} \rightarrow (4, 8) = 32 \text{ values}$
GPU-1: $g_{\text{vocab}}^{\text{(B)}} = \partial \text{Loss}_B / \partial W_{\text{vocab}} \rightarrow (4, 8) = 32 \text{ values}$

FLUSH: Discard W_{vocab} (parameters) |
REDUCE-SCATTER: gradients (like ZeRO-2) |
→ GPU-1 keeps $\text{avg}_{g_{\text{vocab}}}$ (W_{vocab} is in its slice) |
→ GPU-0 discards g_{vocab} immediately |

Backward Layer 3: FFN — Gather, Backprop, Flush

GATHER: All-gather W_1, b_1, W_2, b_2 again	
296 bytes temporary (this is the peak again)	

COMPUTE BACKWARD:

Both GPUs compute local gradients:

$g_{W1}^{(local)} \rightarrow (4, 16) = 64$ values

$g_{b1}^{(local)} \rightarrow (16,) = 16$ values

$g_{W2}^{(local)} \rightarrow (16, 4) = 64$ values

$g_{b2}^{(local)} \rightarrow (4,) = 4$ values

FLUSH: Discard full FFN parameters	
REDUCE-SCATTER: gradients	
→ Each GPU keeps only its averaged gradient slice	
→ GPU-0 keeps avg grads for its portion of W_1	
→ GPU-1 keeps avg grads for rest of W_1, b_1, W_2, b_2	
→ Non-owned gradients discarded immediately	

Backward Layer 2: LN2 — Gather, Backprop, Flush

GATHER: All-gather γ_2, β_2 (16 bytes temporary)

COMPUTE: Backprop through LN2

FLUSH + REDUCE-SCATTER: Discard params, scatter gradients

Backward Layer 1: Attention — Gather, Backprop, Flush

GATHER: All-gather W_q, W_k, W_v, W_o (128 bytes temp) |

COMPUTE BACKWARD:

GPU-0: $g_q^A = \begin{bmatrix} 0.023 & -0.011 & 0.045 & -0.008 \\ -0.031 & 0.019 & -0.007 & 0.014 \\ 0.012 & -0.028 & 0.033 & -0.005 \\ -0.016 & 0.009 & -0.021 & 0.038 \end{bmatrix}$

GPU-1: $g_q^B = \begin{bmatrix} 0.017 & -0.025 & 0.031 & -0.013 \\ -0.009 & 0.041 & -0.018 & 0.006 \\ 0.028 & -0.014 & 0.022 & -0.035 \\ -0.020 & 0.016 & -0.012 & 0.027 \end{bmatrix}$

FLUSH: Discard gathered W_q, W_k, W_v, W_o |

REDUCE-SCATTER: |

→ GPU-0 receives $avg_g_q, avg_g_k, avg_g_v, avg_g_o$ |
(all in its slice) |

$avg_g_q = (g_q^A + g_q^B) / 2$ |
 $= \begin{bmatrix} 0.020 & -0.018 & 0.038 & -0.0105 \\ -0.020 & 0.030 & -0.0125 & 0.010 \\ 0.020 & -0.021 & 0.0275 & -0.020 \\ -0.018 & 0.0125 & -0.0165 & 0.0325 \end{bmatrix}$ |

→ GPU-1 discards all attention gradients |

Backward Layer 0: LN1 — Gather, Backprop, Flush

GATHER: All-gather γ_1, β_1 (16 bytes temporary)

COMPUTE: Backprop through LN1

FLUSH + REDUCE-SCATTER: Discard params, scatter gradients

Backward Pass Complete. Summary of Communication During Backward:

Layer	All-Gather (params)	Reduce-Scatter (grads)
Output	$32 \times 2 = 64 \text{ B}$	$32 \times 2 = 64 \text{ B}$
FFN	$148 \times 2 = 296 \text{ B}$	$148 \times 2 = 296 \text{ B}$
LN2	$8 \times 2 = 16 \text{ B}$	$8 \times 2 = 16 \text{ B}$
Attention	$64 \times 2 = 128 \text{ B}$	$64 \times 2 = 128 \text{ B}$
LN1	$8 \times 2 = 16 \text{ B}$	$8 \times 2 = 16 \text{ B}$
TOTAL	$260 \times 2 = 520 \text{ B}$	$260 \times 2 = 520 \text{ B}$
	↑↑↑	↑↑↑
	NEW (not in ZeRO-2)	Same as ZeRO-2

STEP 3: Optimizer Step (Update Local Slice)

Identical to ZeRO-1 and ZeRO-2. Each GPU runs Adam only on its 130-element slice.

GPU-0 updates W_q (Adam step, iteration $t=1$):

$lr=0.001, \beta_1=0.9, \beta_2=0.999, \epsilon=1e-8$

For $W_q[0,0]$:

Current master param (FP32): $p = 0.12$

Averaged gradient: $g = 0.020$

$m_{new} = 0.9 \times 0.0 + 0.1 \times 0.020 = 0.002$

$v_{new} = 0.999 \times 0.0 + 0.001 \times 0.0004 = 0.0000004$

$\hat{m} = 0.002 / (1 - 0.9^1) = 0.02$

$\hat{v} = 0.0000004 / (1 - 0.999^1) = 0.0004$

$p_{new} = 0.12 - 0.001 \times 0.02 / (\sqrt{0.0004} + 1e-8)$

$= 0.12 - 0.001$

$= 0.119$

Cast to BF16: $W_q[0,0] \approx 0.1191$

GPU-0 repeats for all 130 elements. GPU-1 does the same for its 130 elements.

STEP 4: End of Step — NO All-Gather!

This is another key difference from ZeRO-1/2. There is NO all-gather after the optimizer step.

In ZeRO-1/2, after the optimizer update, we did an all-gather to reconstruct the full parameter set on every GPU. In ZeRO-3, **we don't bother** — each GPU just keeps its updated shard. The full parameters will be

gathered on-demand at the start of the next forward pass, layer by layer.

After optimizer step:

GPU-0: new_params[0:130] in BF16 (updated) → keeps it, that's all

GPU-1: new_params[130:260] in BF16 (updated) → keeps it, that's all

No communication here. The model stays sharded.

STEP 5: End of Step — What Lives Where

GPU-0	
$\theta_0[130]$ in BF16 (parameter SHARD only, updated) = 260 bytes	
—— Optimizer (ONLY slice 0:130) ——	
$m_0[130]$ in FP32 (first moments) = 520 bytes	
$v_0[130]$ in FP32 (second moments) = 520 bytes	
$p_0[130]$ in FP32 (master params) = 520 bytes	
TOTAL AT REST: 1,820 bytes	
(was 2,340 in ZeRO-2)	
(was 2,600 in ZeRO-1)	
(was 4,160 without ZeRO)	

GPU-1	
$\theta_1[130]$ in BF16 (parameter SHARD only, updated) = 260 bytes	
$m_1[130]$ in FP32 (first moments) = 520 bytes	
$v_1[130]$ in FP32 (second moments) = 520 bytes	
$p_1[130]$ in FP32 (master params) = 520 bytes	
TOTAL AT REST: 1,820 bytes	

No gradients stored at rest (freed after optimizer step). **No full parameter replica** (only shards).

7. Communication Cost Analysis — The Price of ZeRO-3

ZeRO-1 ZeRO-2 ZeRO-3

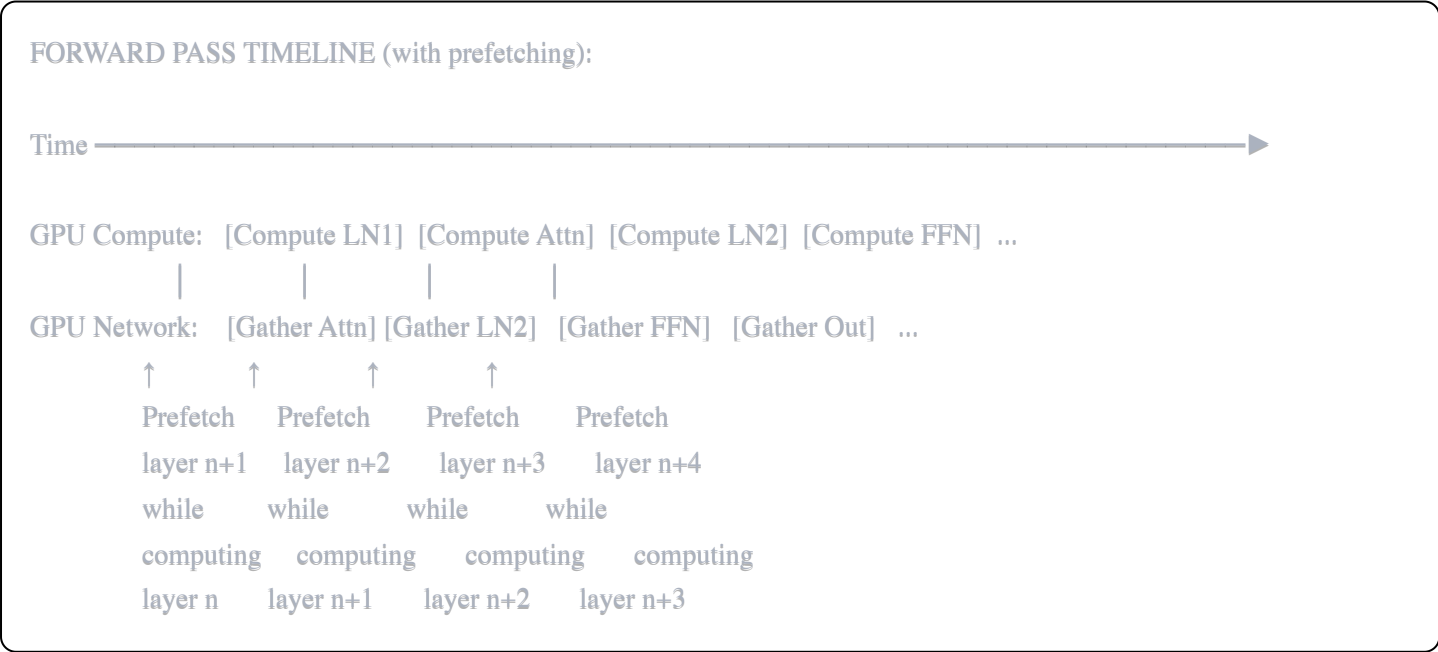
Forward pass:	0	0	520 B ← NEW (all-gather each layer)
Backward pass:			
All-gather params:	0	0	520 B ← NEW
Reduce-scatter:	520 B (fused)	520 B (fused)	520 B
After optimizer:			
All-gather params:	520 B	520 B	0 B ← SAVED (stay sharded)
TOTAL:	1,040 B	1,040 B	1,560 B
Overhead vs ZeRO-2:	—	—	+50%

ZeRO-3 trades 50% more communication for the ability to partition parameters. The extra cost comes from needing to all-gather parameters twice (once for forward, once for backward) instead of the single all-gather after optimizer that ZeRO-1/2 use.

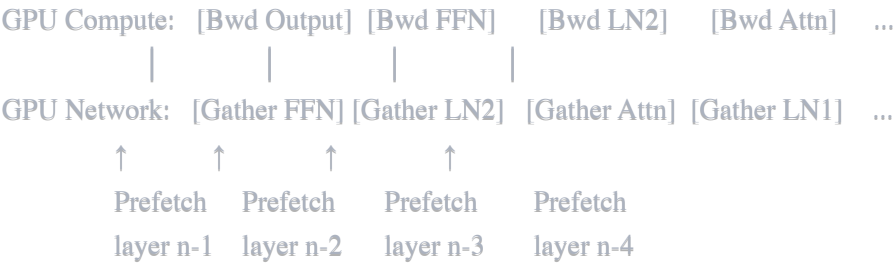
However, in ZeRO-3, the post-optimizer all-gather is eliminated (parameters stay sharded), which recovers some of it. Net extra = 1 additional all-gather of all parameters.

8. Prefetching: How ZeRO-3 Hides the Communication Cost

The all-gathers during forward and backward can be **overlapped with computation** via prefetching:



BACKWARD PASS TIMELINE (with prefetching):



Rule of thumb: prefetching works well as long as the DP (data-parallel) size does not exceed approximately 512 GPUs. Beyond that, the all-gather latency starts to exceed the compute time for a single layer, and the overlap breaks down.

9. Frame-by-Frame Memory During Forward Pass

This shows why ZeRO-3 uses less memory despite the temporary all-gathers:

	ZeRO-2 (GPU-0 memory)	ZeRO-3 (GPU-0 memory)
At rest:		
Params:	260 vals (520 B) FULL	130 vals (260 B) SHARD
Optimizer:	1,560 B	1,560 B
BASELINE:	2,080 B	1,820 B
Computing LN1:		
ZeRO-2:	params already in memory (520 B)	
ZeRO-3:	gather γ_1, β_1 temporarily (260 + 16 = 276 B params)	
	↑ shard + temp full LN1	
Computing Attention:		
ZeRO-2:	params already in memory (520 B)	
ZeRO-3:	gather W_q, W_k, W_v, W_o (260 + 128 = 388 B params)	
	flushed LN1 non-owned	↑ shard + temp full attn
Computing FFN:		
ZeRO-2:	params already in memory (520 B)	
ZeRO-3:	gather W_1, b_1, W_2, b_2 (260 + 296 = 556 B params)	
	flushed attention	↑ PEAK! Briefly exceeds ZeRO-2
	(but only for this layer)	...then immediately flushed
After flush:		
ZeRO-3 returns to	260 B	(just the shard again)

Key insight: ZeRO-3's peak temporary memory during the largest layer briefly approaches (or can slightly exceed) ZeRO-2's steady state. But on average, ZeRO-3 uses significantly less memory because only one layer's full parameters are materialized at a time.

10. Scaling to a Real Model: 7B Parameters on 8 GPUs

	No ZeRO (per GPU)	ZeRO-1 (8 GPUs)	ZeRO-2 (8 GPUs)	ZeRO-3 (8 GPUs)
Params (BF16):	14.0 GB	14.0 GB	14.0 GB	1.75 GB ← +8!
Gradients (BF16):	14.0 GB	14.0 GB	1.75 GB	1.75 GB
Optimizer:				
m (FP32):	28.0 GB	3.5 GB	3.5 GB	3.5 GB
v (FP32):	28.0 GB	3.5 GB	3.5 GB	3.5 GB
master p (FP32):	28.0 GB	3.5 GB	3.5 GB	3.5 GB
TOTAL per GPU:	112.0 GB	38.5 GB	26.25 GB	14.0 GB
Saving vs baseline:	—	65.6%	76.6%	87.5%
Communication: (relative to base)	1× allred	1× allred	1× allred	1.5× allred

14 GB per GPU! A 7B model that originally needed 112 GB per GPU can now train on 8 × 16GB GPUs — that's 8 consumer GPUs.

Temporary peak during the largest layer: ~1.75 GB (shard) + ~2 GB (largest layer gathered) ≈ 3.75 GB of parameter memory, still well within budget.

11. Summary: The Full ZeRO Progression

	Params	Grads	Optimizer	Communication
No ZeRO:	Full (2P)	Full (2P)	Full (12P)	1× (all-reduce)
ZeRO-1:	Full (2P)	Full (2P)	Shard (12P/N)	1× (same)
ZeRO-2:	Full (2P)	Shard (2P/N)	Shard (12P/N)	1× (same)
ZeRO-3 (FSDP):	Shard (2P/N)	Shard (2P/N)	Shard (12P/N)	1.5× (+50%)
Per-GPU memory formula:				
No ZeRO:	2P + 2P + 12P	= 16P		

ZeRO-1: $2P + 2P + 12P/N = (4 + 12/N) \times P$
ZeRO-2: $2P + 2P/N + 12P/N = (2 + 14/N) \times P$
ZeRO-3: $2P/N + 2P/N + 12P/N = 16P/N \quad \leftarrow \text{PERFECT SCALING!}$

For our tiny model (P=260, N=2):

No ZeRO: 4,160 bytes
ZeRO-1: 2,600 bytes (optimizer sharded)
ZeRO-2: 2,340 bytes (+ gradients sharded)
ZeRO-3: 2,080 bytes (+ parameters sharded)

At rest (excluding gradients):

ZeRO-3: 1,820 bytes (truly minimal: shard + its optimizer)

The Tradeoff:

ZeRO-3 gives you:

- ✓ 16P/N memory — perfect linear scaling with GPU count
- ✓ Ability to train models that don't fit in a single GPU's memory at all
- ✓ No full model replica anywhere (true sharding)

ZeRO-3 costs you:

- ✗ 1.5× communication (extra all-gathers during forward and backward)
- ✗ Implementation complexity (gather-compute-flush per layer)
- ✗ Sensitivity to DP group size (prefetching degrades past ~512 GPUs)

When to use which:

- Model fits in GPU memory comfortably → ZeRO-1 (simplest, no comm overhead)
- Memory is tight but model fits → ZeRO-2 (free gradient saving)
- Model does NOT fit in a single GPU → ZeRO-3 / FSDP (the only option)